TransferIa · Documentos de cambios

ZIP con contraseña y una fuente en varias caras

Entrega 10 · Resultados cifrados y la misma cara en varias personas

Commit: `d6b94d6` "ZIP con contraseña, una fuente en varias caras y cuaderno de Colab"
Fecha: 26 de octubre de 2026
Documento generado: 27 de octubre de 2026
Pruebas: caras generadas por IA, en CPU

Contenido

1. Resumen de la entrega 2
2. Una fuente en varias caras 2
2.1 El problema 2
2.2 La causa 3
3. Guardar en ZIP con contraseña 4
3.1 Para qué 4
3.2 Cómo usarlo en la ventana 4
3.3 Cómo usarlo en la CLI 5
3.4 Probado: abrir el ZIP 6
3.5 De qué manera funciona: nunca se pierde el resultado 6
4. Cómo se comprobó 7
Cómo probarlo tú 8
5. Lo que aprendimos 8
Glosario 8

Si el índice aparece vacío, en Word haz clic derecho sobre él → Actualizar campos.

1. Resumen de la entrega

> **Nota:** Todo sale del commit `d6b94d6` del 26 de octubre. Ese commit también agregó el cuaderno de Colab, que no se documenta aquí. Las pruebas usan una copia de esa versión y caras **generadas por IA**.

Cambio	Interfaz	CLI	Qué gana el usuario
Guardar en ZIP con contraseña	Casilla **Guardar en ZIP con contraseña**	`--zip`, `--zip-password`, `--zip-conservar`	Que el resultado no quede suelto: sin la contraseña no se puede ver
Una fuente en varias caras	Índices `0,0` → `0,1`	`--source-index 0,0 --target-index 0,1`	Poner la misma cara a todas las personas de la imagen

2. Una fuente en varias caras

2.1 El problema

Para poner **la misma cara** a varias personas se escribe la fuente repetida: `--source-index 0,0,0 --target-index 0,1,2` ("la cara 0 de la fuente va a las caras 0, 1 y 2 del destino"). Pero solo cambiaba **la primera**.

Cómo reproducirlo

1. Destino: una imagen con tres personas (caras de IA, `tres_personas.jpg`). Fuente: una foto con una sola cara.
2. Con la versión anterior (`672f82a`): `python cli.py process --source fuente.jpg --target tres_personas.jpg --output antes.jpg --source-index 0,0,0 --target-index 0,1,2`

3. Mismo comando con la versión nueva (`d6b94d6`).



Figura 1. *Arriba el destino; al medio la versión anterior; abajo la nueva, con la misma fuente en las tres caras*

Qué hay que mirar: en la fila del medio solo cambió la persona de la izquierda. Medido como diferencia media de cada tercio de la imagen con el original: antes **5,3 / 0,2 / 0,1** (las dos de la derecha intactas); ahora **5,3 / 9,5 / 5,2** (las tres cambiaron).

2.2 La causa

El bucle recorría tantos pares como el **mínimo** entre cuatro cantidades, una de ellas el número de caras de la fuente. Con una fuente de una sola cara, el mínimo era 1: se cambiaba un par y el bucle terminaba.

Antes (core/Face_Swaper.py)

```
for i in range(min(len(caras_src), len(caras_dst), len(indices_src), len(indices_dst))):
    indice_src = indices_src[i]
    indice_dst = indices_dst[i]
    ...
```

Después

```
for indice_src, indice_dst in zip(indices_src, indices_dst):
    if not (0 <= indice_src < len(caras_src) and 0 <= indice_dst < len(caras_dst)):
        continue          # en este frame no está esa cara: se salta el par
    ...
```

Ahora se recorre **cada par** de índices (`zip` los toma de a dos: 0→0, 0→1, 0→2). La cantidad de caras de la fuente ya no limita nada: solo se comprueba que cada índice exista. Si en un frame de video falta una cara pedida, ese par se salta y los demás siguen.

> **Para aprender:** Un error de este tipo se llama "límite mal elegido": la cantidad de vueltas dependía de algo (las caras de la fuente) que no tiene por qué coincidir con lo que se quiere recorrer (los pares pedidos). Recorrer directamente la lista de pares lo evita.

3. Guardar en ZIP con contraseña

3.1 Para qué

Los resultados son caras de personas. Guardarlos en un **ZIP cifrado** hace que, si alguien abre la carpeta de salida o se copia el archivo, no pueda ver la imagen o el video sin la contraseña.

Se usa cifrado **AES-256** (el estándar actual) con la biblioteca `pyzipper`. Se descartó el cifrado clásico de ZIP ("ZipCrypto"), que se puede romper con herramientas públicas. A cambio, el Explorador de Windows no abre ZIP con AES: hay que usar **7-Zip** o **WinRAR**.

3.2 Cómo usarlo en la ventana

1. En **Configuración de Salida**, marca **Guardar en ZIP con contraseña**. Se activan el campo **Contraseña** y **Conservar también el archivo suelto** (sin la casilla quedan en gris).
2. Escribe la contraseña. Se ve con puntos (•) y **no se guarda** en ningún lado, ni en las configuraciones.
3. Pulsa **Iniciar Procesamiento**. Al terminar verás dónde quedó el ZIP.

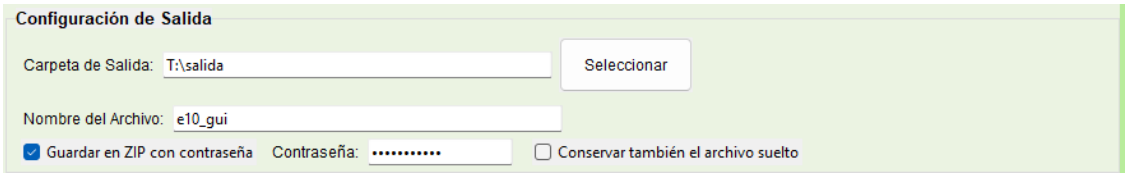


Figura 2. *La fila nueva en "Configuración de Salida", con la contraseña escrita*

Si marcas la casilla pero no escribes la contraseña, avisa antes de empezar y no procesa nada:

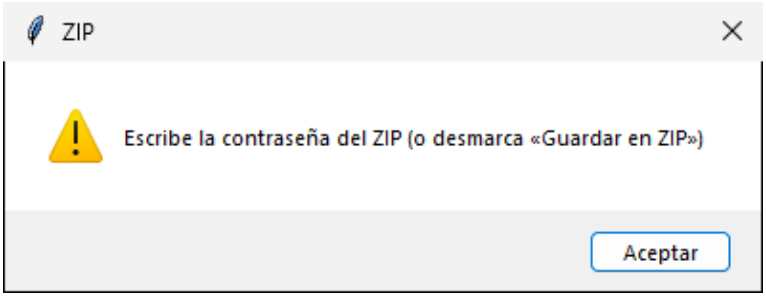


Figura 3. *Aviso al procesar con la casilla marcada y sin contraseña*

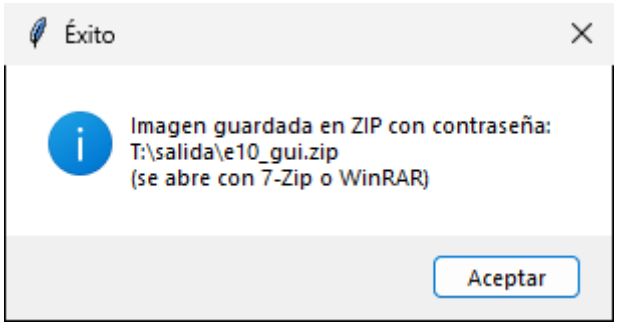


Figura 4. *Mensaje al terminar: el ZIP quedó en la carpeta de salida*

3.3 Cómo usarlo en la CLI

Opción	Qué hace
`--zip`	Guarda el resultado en `<salida>.zip` y borra el archivo suelto. La contraseña se **pregunta** al empezar, dos veces y sin mostrarla.
`--zip-password CLAVE`	Da la contraseña en el comando, sin preguntarla. Queda en el historial de la consola: mejor no usarla.
`--zip-conservar`	Además del ZIP, deja el archivo suelto.

```
> python cli.py process --source fuente.jpg --target destino.jpg --output salida/e10_zip.jpg --zip -
  -zip-password Prueba-2026
Resultado guardado en: <prueba>\salida\e10_zip.zip (con contraseña; se abre con 7-Zip o WinRAR)

> python cli.py process --source fuente.jpg --target carpeta_fotos --output salida --zip --zip-passw
  ord Prueba-2026
3 imagen(es) -> <prueba>\salida\carpeta_fotos_output
Imagenes: [##########----------]  33.33% (1/3)
Imagenes: [####################----------]  66.67% (2/3)
Imagenes: [##############################] 100.00% (3/3)
3 de 3 imagen(es) guardada(s)
Resultado guardado en: <prueba>\salida\carpeta_fotos_output.zip (con contraseña; se abre con 7-Zip o
   WinRAR)

> python cli.py process --source fuente.jpg --target destino.jpg --output salida/e10_cons.jpg --zip
  --zip-password Prueba-2026 --zip-conservar
Resultado guardado en: <prueba>\salida\e10_cons.zip (con contraseña; se abre con 7-Zip o WinRAR)
Tambien se conservo: <prueba>\salida\e10_cons.jpg

> dir /b salida  (solo lo de esta prueba)
e10_zip.zip
carpeta_fotos_output.zip
e10_cons.jpg
e10_cons.zip
```

Figura 5. *Imagen, carpeta y "conservar"; y lo que queda en la carpeta de salida*

Con una **carpeta** de imágenes como destino, todas van a un solo ZIP (`carpeta_fotos_output.zip`) dentro de una carpeta con ese nombre, y la carpeta suelta se borra solo si quedó vacía.

3.4 Probado: abrir el ZIP

Se abrió el ZIP con `pyzipper`: con una letra cambiada no abre; con la contraseña correcta, sí. El módulo `zipfile` de Python, que no conoce AES, no puede leerlo.

> **Nota:** No probado con 7-Zip ni WinRAR: no están instalados en el PC de pruebas. El formato es el estándar WinZip AES, que ambos programas leen.

```
> python abrir_zip.py salida/e10_zip.zip Prueba-2026
Contenido de e10_zip.zip:
  e10_zip.jpg                    57,907 bytes  cifrado AES, sin comprimir
Con la contraseña 'prueba-2026': Bad password for file 'e10_zip.jpg'
Con la contraseña 'Prueba-2026': se abre (57,907 bytes de e10_zip.jpg)

> python -c "zipfile.ZipFile('salida/e10_zip.zip').read('e10_zip.jpg', pwd=b'Prueba-2026')"
NotImplementedError: That compression method is not supported
```

Figura 6. *Contenido del ZIP, contraseña equivocada y correcta*

3.5 De qué manera funciona: nunca se pierde el resultado

La idea central: **no se borra nada hasta comprobar que el ZIP está bien**. Si el disco se llena o algo falla a la mitad, el resultado sigue suelto en la carpeta.

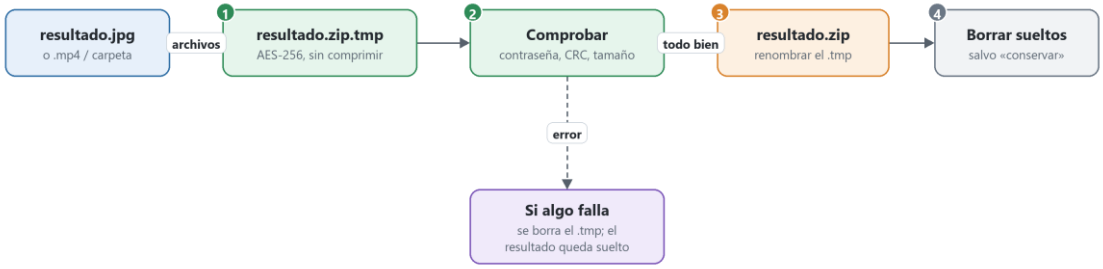


Figura 7. *Pasos de empaquetar() en core/Empaquetar.py*

1. Escribe un ZIP **temporal** (`.zip.tmp`) con AES-256. Los archivos van **sin comprimir** (`ZIP_STORED`): JPG y MP4 ya vienen comprimidos, así que comprimir otra vez solo tarda más y no achica.
2. Lo vuelve a abrir con la contraseña y comprueba cada archivo: su **CRC** (un número de control que cambia si un solo byte está mal) y su tamaño.
3. Solo si todo está bien, renombra el `.tmp` a `.zip`.
4. Borra los archivos sueltos, salvo que se pidió conservarlos.

```
with pyzipper.AESZipFile(temporal, "w", compression=zipfile.ZIP_STORED, encryption=pyzipper.WZ_AES) as zip_:
    zip_.setpassword(clave)
    zip_.setencryption(pyzipper.WZ_AES, nbits=256)
    for archivo, nombre in zip(archivos, nombres):
        zip_.write(archivo, nombre)
# Verificacion: se abre con la contrasena, cada archivo pasa su control (CRC) y tiene su tamaño
with pyzipper.AESZipFile(temporal) as zip_:
    zip_.setpassword(clave)
    if zip_.testzip() is not None:
        raise RuntimeError("el ZIP quedó dañado")
...
os.replace(temporal, destino)       # recién aquí aparece el .zip
```

El `finally` del código borra el `.tmp` pase lo que pase, así no quedan ZIP a medias. Si el ZIP ya existe, la CLI no lo pisa sin `--overwrite`:

```
> python cli.py process --source fuente.jpg --target destino.jpg --output salida/e10_zip.jpg --z
  ip --zip-password Prueba-2026
Error: El ZIP ya existe; usa --overwrite: <prueba>\salida\e10_zip.zip
```

Figura 8. *Segundo intento con la misma salida*

> **Advertencia:** Si olvidas la contraseña, **no hay forma de recuperar** el contenido del ZIP. Por eso, la primera vez, prueba con **Conservar también el archivo suelto** hasta confirmar que lo abres con 7-Zip.

> **Nota:** La contraseña y la casilla "conservar" no cuentan para la "firma" de un video a medias: cambiar la contraseña no impide continuar un video cortado (`_OPCIONES_SIN_EFECTO` en `cli.py`).

4. Cómo se comprobó

- Tres personas con la misma fuente, en la versión anterior y en la nueva; se midió cuánto cambió cada tercio de la imagen.
- ZIP de una imagen, de una carpeta de 3 imágenes y con `--zip-conservar`; se listó la carpeta de salida.
- Se abrió el ZIP con contraseña equivocada y correcta, y con `zipfile` (sin AES).
- En la ventana: casilla sin contraseña (aviso) y con contraseña (mensaje final).

Cómo probarlo tú

1. Instala la dependencia: `pip install pyzipper` (ya está en `requirements.txt`).
2. Procesa una imagen con **Guardar en ZIP con contraseña** marcado.
3. Abre el `.zip` con 7-Zip: pide la contraseña.

5. Lo que aprendimos

- Antes de borrar el original, comprobar la copia: escribir aparte, verificar y recién después renombrar y borrar.
- No inventar cifrado: usar el estándar (AES-256) con una biblioteca conocida.
- Una contraseña escrita en un comando queda en el historial; es mejor pedirla sin mostrarla (`getpass`).
- Recorrer los pares pedidos, no un mínimo de cantidades que no tienen que ver.

Glosario

Término	Significado
AES-256	Cifrado simétrico estándar con clave de 256 bits; sin la contraseña no hay forma práctica de leer el contenido.
ZipCrypto	El cifrado viejo de los ZIP; débil, se rompe con herramientas públicas.
CRC	Número de control calculado sobre el contenido; si cambia un byte, cambia el CRC.
ZIP_STORED	Guardar dentro del ZIP sin comprimir.
getpass	Función de Python que lee una contraseña sin mostrarla en pantalla.